

OT/ICS Lab — MASTER

OT/ICS Security Lab — MASTER (Status & Index)

Single source of truth for this project's status, decisions, risks, and document map. Vendor-neutral OT/ICS security lab — a portfolio / recruiter-facing artifact where the *reasoning* behind each choice is part of the deliverable, not just the plumbing.

- **Project:** Vendor-neutral OT/ICS security lab (portfolio / recruiter-facing)
- **Process simulated:** Generic plant tank-fill (level, pump, valve, high alarm)
- **Phase:** Phases 1-3 complete. Queue Item 1 (historian stack) and Queue Item 3 (runbook expansion) shipped 2026-05-27. Queue Item 2 (attack-execution docs against the v4 stack) remaining.
- **Phase 1 status: Acceptance gate fully met — runtime, visual HMI, and Modbus/TCP capture all proven end-to-end** (see Acceptance Gate)
- **Last updated:** 2026-05-27
- **Repo:** <repo-root> (your local clone location; absolute path is environment-specific)

Document Index

File	Purpose
README.md	What it is, prerequisites, setup, run, test, screenshots, security talking points
docs/architecture.md	Components, control + process model, data flow, zones, trust boundaries
docs/network-security.md	Threat model, supply-chain control, least privilege, segmentation, exposure, attacker view
docs/monitoring.md	Wireshark Modbus capture (host + container paths), normal baseline vs. suspicious
docs/MASTER.md	This file — status, locked decisions, risks, Phase 2 and Phase 3 backlogs
plc/Dockerfile	OpenPLC v3 built from pinned upstream source (supply-chain rationale in header)
plc/tank_fill.st	Control logic + process model, IEC 61131-3 Structured Text
docker-compose.yml	2 services, 3 networks, FUXA persistence
.gitignore	Runtime state + captures excluded

Phase 1 — Deliverable Checklist

- ☒ **1. Folder structure** — plc/ fuxa/appdata/ docs/ captures/
- ☒ **2. Compose stack**
 - ☒ 2a plc/Dockerfile — OpenPLC v3 built from pinned upstream source
 - ☒ 2b docker-compose.yml — 2 services, 3 networks, FUXA persistence
 - ☒ 2c .gitignore — runtime state + captures excluded
- ☒ **3. plc/tank_fill.st** — OpenPLC tank-fill control logic (Structured Text)
- ☒ **4. README.md** — what it is, prereqs, setup, run, test, screenshots, talking points
- ☒ **5. docs/architecture.md** — components, data flow, zones
- ☒ **6. docs/network-security.md** — segmentation rationale, exposure, attacker view
- ☒ **7. docs/monitoring.md** — Wireshark Modbus capture, normal vs. suspicious
- ☒ **8. docs/MASTER.md** — full index + final status (this file; stub replaced)

All 8 build deliverables are authored. This is *not* the same as Phase 1 acceptance — see the gate below.

Acceptance Gate (Phase 1 — fully met)

All four Phase 1 acceptance criteria are met:

- ☒ docker compose up --build brings up otlab-openplc + otlab-fuxa
- ☒ tank_fill.st compiles in OpenPLC and the PLC is **Running** (clean compile after the MATIEC fix-pass — see Known Risks)
- ☒ FUXA connects to openplc:502 (Slave ID 1, status green) and the tank cycles per the hysteresis logic — data path verified via pymodbus from a container on otlab_ot_zone (HR0 cycled 49→69→71→51→31; HR1/HR2 = SP_Low 20 / SP_High 80), and the FUXA HMI (bar gauge bound to Level, numeric % label, Pump ON/OFF, Valve OPEN/CLOSED, Alarm NORMAL/ACTIVE indicators) renders the live cycling — screenshot at docs/img/fuxa-hmi.png shows tank at 68 % with a manually-triggered alarm so all three indicator states are visible in one frame.
- ☒ Modbus/TCP packets captured in Wireshark per docs/monitoring.md — capture taken from inside otlab-openplc on the ot_zone bridge: 2502 packets, 0 dropped, steady FC 3 + FC 1 polling pattern from FUXA (172.19.0.3) → OpenPLC (172.19.0.2:502), no writes (FC 5/6/15/16 = 0), no Modbus exceptions, ~1 s cadence, single long-lived TCP connection. Matches docs/monitoring.md §4 normal baseline exactly. Capture at captures/modbus-baseline-20260520.pcap (gitignored); screenshot at docs/img/wireshark-modbus.png.

Status: Phase 1 fully accepted in this environment — OpenPLC built from pinned source, ST compiled clean, PLC Running, live Modbus/TCP serving correct cycling data on :502, FUXA HMI animating with all six bound tags (Level HR0, SP_Low HR1, SP_High HR2, Pump Coil 0, Valve Coil 1, Alarm Coil 2), and the Modbus/TCP capture confirms the clean normal-poll baseline documented in docs/monitoring.md §4 (reads-only, single client, single long-lived connection). Ready for public repo flip.

Key Decisions Locked (recruiter-relevant)

Decision	Choice	Why it matters
OpenPLC image	Built from pinned upstream commit b5d4135...	No unvetted Docker Hub image — supply-chain control is part of the security story
FUXA image	frangoteam/fuxa:1.3.1 (pinned)	No moving :latest; reproducible
--privileged	Not set on OpenPLC	Deliberate least-privilege; no hardware I/O in this lab
.env file	None in Phase 1	Fewer first-run failure modes for a beginner
Zoning	ot_zone, mgmt_zone, attacker_zone	attacker_zone is the genuinely enforced boundary — proof landed 2026-05-22 (Phase 2 Findings §1)
Suricata capabilities	NET_ADMIN + NET_RAW only (intentionally no SYS_NICE)	Permits :ro bind mounts on rules + config (supply-chain control: Suricata cannot tamper with its own ruleset at runtime even if compromised). Trade: cosmetic “running as root” startup warning, accepted — root inside a non-privileged container with no host access is not a security issue
FUXA Modbus healthcheck	Detection-only (no autoheal sidecar)	FUXA 1.3.1 ModbusTCP driver has no auto-reconnect (Finding §10). Healthcheck shipped at 626fc56 marks fuxa as (unhealthy) within ~90s of a silent drop; operator manually runs docker compose up -d --force-recreate fuxa to recover. Local-use lab; autoheal sidecar deferred to portfolio-prep phase post-cert

Known Risks / VERIFY-ON-FIRST-RUN

- OpenPLC builds from source on first docker compose up (~minutes; unverified end-to-end in this environment) — fallback documented in README.
- OpenPLC running without --privileged — fallback documented in README.
- FUXA persistence path /usr/src/app/FUXA/server/_appdata — confirmed on first run.
- ~~OpenPLC Structured Text program may need a fix pass on first compile~~ **RESOLVED 2026-05-19**. MATIEC rejected mixing AT-located and non-located declarations in one VAR block (errors lines 45-54, cascading to 6 assignments). Fix: split into two homogeneous VAR...END_VAR blocks within the same PROGRAM (located outputs separate from internal state). Clean compile, PLC Running.
- **OpenPLC v3 web UI Monitoring page is unreliable** — shows zeros and only 3 of 6 variables despite correct live Modbus output. Cosmetic UI quirk of OpenPLC v3, not a runtime fault. Modbus/TCP (pymodbus / Wireshark) is the source of truth. Dashboard status display also caches stale state: after Start PLC *and* after docker compose down/up cycles, it can show the wrong run/stop indicator even with Ctrl+F5. Authoritative “is the PLC actually running” answer is docker logs otlab-openplc or a Modbus read against :502 (e.g., pymodbus reading HR0 — non-zero, cycling values prove the PLC is running), never the dashboard.
- **OpenPLC startup crash if Programs table and active_program pointer drift apart** — observed 2026-05-21 after a Docker Desktop restart cycle. The named volume preserved openplc.db, but the Programs table row for the active program was missing while /opt/OpenPLC_v3/webserver/active_program still pointed at 927367.st. At HTTP server startup webserver.py:2726 runs SELECT * FROM Programs WHERE File = '<active_program>'; with no row it returns None, and the run_http thread dies with TypeError: 'NoneType' object is not subscriptable. The web UI (:8080) is then unreachable and the Modbus runtime never starts (it needs a loaded program). Recovery — restore the missing row from inside the container. The underlying cause is OpenPLC v3's lack of transactional linkage between the pointer file and the database row, not a bug in the lab; the recovery below is verified persistent across a subsequent full docker compose down/up -d cycle (curl -I http://localhost:8080 returns HTTP/1.1 302, FUXA reconnects):

```
# 1. Read the pointer to get the active program filename
docker exec otlab-openplc cat
/opt/OpenPLC_v3/webserver/active_program

# 2. Insert the matching Programs row (substitute the filename
from step 1)
docker exec otlab-openplc sqlite3
/opt/OpenPLC_v3/webserver/openplc.db \
    "INSERT INTO Programs (Name, Description, File, Date_upload)
VALUES \
    ('Tank Fill', 'Generic plant tank-fill process', '<filename>',
strftime('%s','now'));"

# 3. Restart so OpenPLC re-reads state
docker compose restart openplc
```

- FUXA → OpenPLC Modbus host must be the service name openplc:502, never localhost — the #1 first-run failure (fix in README Troubleshooting).
-

Authoring Discipline (lessons learned)

Doc references must resolve. When any doc in this repo names a file path (image, log, config, script), that path must either exist on disk at the time of commit OR be explicitly flagged as MANUAL with a clear “you need to create this” note. Authoring references to non-existent paths produces the silent drift the project rules exist to prevent.

- Lesson source: 2026-05-19 MASTER.md update referenced docs/img/fuxa-hmi.png in two places before the docs/img/ directory was created — caught and fixed same day, but is exactly the class of error this rule blocks.
- Process improvement (future projects): when a .gitignore is authored at folder creation, git init should happen in the same step. Lesson source: 2026-05-19 — this repo had a Phase-1-anticipating .gitignore from day one but git init only happened on day three, meaning every prior fix pass (MATIEC, HMI, MASTER.md updates) shipped without revision history.
- 2026-05-20 — pre-public-flip scan caught two hardcoded absolute paths in committed docs (README.md, docs/MASTER.md), both pointing at the author’s local WSL2 clone location. Generalized to a <repo-root> placeholder, defined inline at each site. Process improvement: before any first push to a public repo, grep for user-specific paths and known sensitive strings as a standard pre-flight.
- 2026-05-20 — Phase 1 compose stack persisted FUXA HMI state but not OpenPLC program/runtime state. Asymmetry discovered during a routine down/up cycle test. Process improvement: when one service in a stack has persistence wired, audit every other service for the same need at the same time, not later.
- 2026-05-21 — Persistence of a named volume kept the data, but OpenPLC’s own startup code is brittle if internal state files (active_program pointer, Programs DB rows) drift out of sync. Process improvement: persistence alone is insufficient — for stateful services with multi-file consistency requirements, document the recovery procedure as a Known Risk, not as a tribal-knowledge incident response.
- 2026-05-21 — a verification spike on an unfamiliar Docker image (OpenPLC v4) caused Docker Desktop to terminate the running Phase 1 containers to free memory for the new image’s startup. Phase 1 recovered cleanly in 8 seconds — the previous day’s named-volume persistence work was inadvertently stress-tested and passed (live Modbus value within one verification cycle). Process improvement: before pulling or running a large unfamiliar Docker image on a Windows/WSL2 host, either raise Docker Desktop’s memory limit first, or docker compose stop the existing stack to give the new image headroom — resilience to a forced shutdown is not a license to invite one.
- 2026-05-23 — Verifying CLI flags against --help before using non-canonical long forms saves a failed run. Suricata accepts only -l <dir> for log directory; --log-dir is not exposed as a long alias. My initial replay used --log-dir — Suricata’s parser silently ignored the unknown flag, fell through to help text, exited 0; output dir was never created and eve.json was never produced. Process improvement: for any tool where the short form is the documented standard, run <tool> --help | grep -i <option-name> before using the assumed long form. Discipline applies broadly, not just to Suricata.
- The discipline above is codified as a user-level Claude Code skill at ~/.claude/skills/verifying-cli-flags/SKILL.md — triggers when about to use a long-form CLI flag (--flag) not yet confirmed in this project’s history or the tool’s --help output. Portable across projects (lives in user-level config, not the repo). RED-phase tested 2026-05-23 against two subagent scenarios; skill fired correctly on iptables long-form flag, stayed silent on a canonical git command.
- 2026-05-24 — **Verify runtime claims against current state, not**

against prior docs. When a doc describes runtime state (IPs, container state, file contents, version pinning, etc.), verify against current state via live inspection (`docker inspect`, `ls`, `cat`) or commit history before propagating claims from prior docs. Prior docs may be stale relative to commits that landed after them. Lesson source: 2026-05-24 caught a triple-stale claim — the 3a evidence doc (shipped 2026-05-23 at b7d9661) said “Suricata is currently not pinned.” Pinning shipped hours later at a91f5e2 but the 3a doc was never updated. When writing 3b’s evidence doc on 2026-05-24, I inherited the stale claim by referencing the 3a doc instead of running `docker inspect otlab-suricata`. The stale framing propagated into project memory and into the next-day prompt for “task 3 — pin Suricata’s IPs.” Corrected at 956ce3d (in-place doc fixes for both 3a and 3b).

- 2026-05-24 — **Tool selection at project start / phase boundary requires a candidates evaluation before building.** Minimum 3 candidates per role, evaluated for: last commit date, deprecation status, open issue volume, known limitations (search "{tool} reconnect/limitations/deprecated"), what real industry uses. Output as a decision-matrix doc in the repo. **Upstream-deprecated = hard stop unless explicit justification with migration plan.** This is a gate, not a suggestion. Lesson source: retrospective on Phase 1 tool choices. OpenPLC v3 was chosen as the PLC by default and is upstream EOL — only discovered during the 2026-05-21 verification spike on v4 (Phase 3 backlog now carries a full v3→v4 migration as a dedicated phase). FUXA 1.3.1’s missing Modbus auto-reconnect (Finding §10) was discovered mid-Phase-2 only when the HMI silently froze. Both would have been caught by a pre-Phase-1 candidates evaluation. The cost of a 1-hour evaluation up front would have been much smaller than the cost of working around these limitations mid-phase.
-

Open Items (Phase 1 follow-ups — not blocking acceptance)

- FUXA HMI layout polish — numeric % label floats per value width (33% / 48% / 68% shift horizontally), minor alignment between the Pump/Valve/Alarm indicator row. Acceptable for Phase 1; track as a Phase 2 polish item, non-blocking.
-

Phase 2 Findings

Live security findings as Phase 2 work progresses. Each finding is dated and tagged with the task that produced it. Pattern matches Phase 1 network-security §5.3’s “prove the boundary, then prove what crossing it costs” — first the boundary holds, then later tasks intentionally cross it to document what becomes possible.

1. `attacker_zone` → `ot_zone` boundary holds (2026-05-22)

Source: Phase 2 step 1 — attacker container brought up on `attacker_zone`.

Setup. New container `otlab-attacker` attached to `attacker_zone` only. Inside the container: one non-loopback interface (`eth0` on 172.20.0.2/16 — the auto-assigned `attacker_zone` bridge), no second NIC, no route to `ot_zone`’s subnet. Outside: `docker inspect` confirms single-network attachment.

Test 1 — name resolution. ping openplc from inside the attacker.
Result: ping: openplc: Name or service not known (exit 2). Docker's embedded DNS is scoped per bridge — openplc is registered on ot_zone, not on attacker_zone, so name resolution itself fails.

Test 2 — TCP reach. TCP knock to openplc:502 (the Modbus port) from inside the attacker, with a 3-second cap:

```
docker exec otlab-attacker bash -c \  
"timeout 3 bash -c '</dev/tcp/openplc/502' && echo OPEN || echo  
BLOCKED"
```

Result: BLOCKED after the 3-second timeout — no route, no peer.

Evidence quality. Phase 1 services (otlab-openplc, otlab-fuxa) were running on ot_zone at the time of the test (5+ minutes uptime). The failure is therefore not “openplc was down anyway” — it is “openplc was alive and serving Modbus on ot_zone, but attacker_zone could not reach it.”

Interpretation. Docker bridge-network isolation alone is sufficient to prevent an attacker_zone container from addressing ot_zone — at both the resolution layer (DNS scoped per bridge) and the routing layer (no route between bridges). This is the architectural claim made in docs/network-security.md and README talking point #4, now empirically demonstrated. Phase 2 step 2's Step 4 verification (2026-05-22) added a direct-IP probe (172.18.0.2:502 from attacker_zone) that also returned BLOCKED — the two layers are therefore independent failures, not a single combined one.

Resolved in Phase 2 step 2 (2026-05-22). The three deferred tests were executed in the attack-scenarios task:

- **Direct-IP reach** is independently blocked. Step 4 verification probes 172.18.0.2:502 (openplc's ot_zone IP) from attacker_zone and gets BLOCKED — same as the name-based probe. The boundary's two layers (DNS scoped per bridge + inter-bridge routing absent) are therefore independent failures, both confirmed.
- **Positive control** is implicit in scenarios 1-5: once the attacker is attached to ot_zone, everything works. The \$1 boundary is real, not a misconfigured test environment.
- **Modbus writes that succeed once the boundary is crossed** — empirical inventory now in Findings \$5 and \$7. The docs/network-security.md \$5.3 scope-honesty note is fully closed across all four output write FCs.

2. mgmt_zone vs ot_zone — what's actually on the wire (2026-05-22)

Source: Pre-flight of the Phase 2 attack-scenarios task (the baseline pcap surfaced this; cited in scenarios 1 and 5 setup).

Summary: The Phase 1 architecture story frames ot_zone as the process/Modbus network. Empirically, the running FUXA↔OpenPLC Modbus poll rides mgmt_zone (172.19.0.x), not ot_zone (172.18.0.x). Both services are multi-homed; Docker's embedded DNS happened to resolve openplc to its mgmt_zone IP when FUXA opened its long-lived connection, and that connection has persisted on mgmt_zone since. docs/network-security.md \$4's honest-scope note (mgmt_zone↔ot_zone is not a hard boundary) covers this; the practical consequence is that the legitimate poll path in this lab's current runtime state is mgmt_zone-only.

Implication: Suricata sensor placement (Phase 2 backlog item) must choose ot_zone vs mgmt_zone vs both. ot_zone-only would miss the legitimate FUXA baseline; mgmt_zone-only would miss attacker traffic

after a boundary cross. Phase 3 mgmt_zone↔ot_zone hardening also needs to know which network the live legitimate traffic actually traverses today.

3. Recon on ot_zone is unimpeded; OpenPLC web UI also serves HTTPS on 8443 — a documentation-drift hardening trap (2026-05-22)

Source: Scenario 1 — docs/phase2/scenarios/scenario-1-recon-nmap.md.

Summary: nmap host discovery + service detection from an ot_zone-attached attacker completes in ~2 min and surfaces every documented service. **A previously undocumented port — openplc:8443 — is also open**, running the same Flask/Werkzeug webserver as the documented :8080 OpenPLC web UI (matching Server: Werkzeug/2.3.7 Python/3.13.5 header, same python3 webserver.py PID, same application). HTTPS via Flask's adhoc SSL context — a second access path to the OpenPLC management UI. The Phase 1 host-published port table (docs/network-security.md §5.1) lists only :8080. HTTP listeners (8080, 1881) also leak framework + version information in Server: headers.

Implication: Documentation-drift hardening trap. Phase 3 work that tightens management access by restricting :8080 (firewall, network policy, mgmt_zone segregation) will leave :8443 wide open as an equivalent attack surface — and the Phase 1 docs give the team no reason to know it exists. Hardening controls on the OpenPLC management UI must treat 8080 + 8443 as one logical surface, or the second port must be documented and explicitly disabled. The generic pattern — *documented port set ≠ actually-listening port set* — is worth a fresh port-scan sanity check at the start of any Phase 3 hardening pass, not just for OpenPLC.

4. Modbus has no access control across all 8 FCs; out-of-map reads return silent zeros, not exceptions (2026-05-22)

Source: Scenario 2 — docs/phase2/scenarios/scenario-2-modbus-fc-scan.md.

Summary: All eight standard Modbus function codes (FC 1, 2, 3, 4, 5, 6, 15, 16) accepted by OpenPLC v3 with no exception. Reads at unmapped addresses (HR 100, HR 1000, coil 100) returned silent zeros — **NOT** Modbus exception 02 (Illegal Data Address) as ICS hardening guidance leads one to expect. Out-of-map writes also accepted (no .st variable backs those addresses, so PLC behavior is unchanged).

Implication: Directly changes the Suricata rule design for docs/monitoring.md §5 row 6 ("Reads outside the map"). The detection signal CANNOT be Modbus exception responses — exceptions never fire here. Suricata must parse address fields out of the Modbus PDU and compare against the documented map. This is the single most actionable Phase 2 finding for the Suricata work in step 3.

5. §5.3 reassertion property extended to all output Modbus objects across all four write FCs (2026-05-22)

Source: Scenarios 3, 4, 4b together — docs/phase2/scenarios/scenario-3-coil-writes.md, scenario-4-hr-writes.md, scenario-4b-fc15-coils.md.

Summary: Phase 1 docs/network-security.md §5.3 documented the reassertion-every-scan property for HR 0 only and explicitly deferred coil writes and FC 15/16 + HR 1/HR 2 to Phase 2. All four output write FCs (FC 5, FC 6, FC 15, FC 16) now empirically tested against all mapped coils (0, 1, 2) and holding registers (1, 2). Architectural property is identical in every case: write accepted at the protocol layer, PLC overwrites within ≤ 1 scan (~ 100 ms). The .st program's internal SP_Low / SP_High constants — which drive the control loop — are NOT Modbus-mapped and remain unreachable from any network peer. The original task hypothesis “could redirect pump behavior” is empirically refuted.

Implication: §5.3 scope-honesty note is closed. Phase 1's refined-framing claim — “attacker on ot_zone can observe everything and can falsify the HMI's view while actively writing, but cannot redirect the PLC's control decisions” — now has full empirical backing.

6. Operational deception confirmed visually on the FUXA HMI (2026-05-22)

Source: Scenario 3 — docs/phase2/scenarios/scenario-3-coil-writes.md; screenshot at docs/img/phase2-scenario-3-alarm-write.png.

Summary: During a 3-second FC 5 burst writing TRUE to the alarm coil, FUXA's HMI rendered the falsified state visibly — tank at 72 % with the Alarm indicator ACTIVE (red), well below the 90 % alarm threshold. FUXA's ~ 1 Hz polling caught a between-scan window where the attacker's write was the most recent value. An operator monitoring this HMI would respond to a phantom alarm — the attack creates real-world impact (investigation, possible process halt, loss of trust in indicator) without changing actual plant behavior.

Implication: The detection signal at this layer must be **source IP + FC**, not state. A rule that watches the coil's *value* will see legitimate alarm activations and cannot distinguish them from this attack. docs/monitoring.md §5 row 2 (new source IP to :502) is the right primitive. Scenario 4's HR-write equivalent is the same class but not visually demonstrated because the current HMI does not display SP_Low / SP_High (tracked in Open Items as HMI polish).

7. Modbus has no authentication and no integrity protection (2026-05-22)

Source: Scenario 5 — docs/phase2/scenarios/scenario-5-modbus-replay.md.

Summary: Two independent demonstrations from one captured PDU. **(1) No authentication:** verbatim replay of a captured FUXA poll (TID 0x00a3) from the attacker's source IP produced a normal PLC response with the original TID echoed — the PLC cannot distinguish the replay from a legitimate request. **(2) No integrity:** a three-byte mutation of the same captured PDU (FC 3 $\rightarrow$ FC 6, address and value bytes reinterpreted) turned a read into a write to HR 1; the PLC accepted the mutated request and acknowledged the write. The §5.3 reassertion wall still applied — the falsified value was overwritten within ~ 1 scan.

Implication: Strengthens the §6 detection conclusion — payload content (TID, FC, address, value) is freely mutable on the wire, so no payload-content-based Suricata signal can distinguish attacker traffic from a clever replay. Source IP + FC is the only signal that works. Defense at the protocol layer is therefore **segmentation + behavioral detection**, not Modbus hardening (which the protocol does not support).

8. Suricata rate-threshold rules are inherently non-deterministic under multi-threaded evaluation (2026-05-23)

Source: Phase 2 sub-session 3a (Suricata recon detection) — docs/phase2/detection/3a-recon-detection.md. Evidence preserved at suricata/logs/replay-3a-verify-3/eve.json (gitignored; local-only).

Summary: Suricata's threshold-mode rules (threshold: type both, track by_src, count N, seconds W) produce non-deterministic alert counts when running over the same input pcap under multi-threaded evaluation. SID 9000002 (TCP SYN scan rate threshold, count 10, seconds 10) was replayed five times against the same scenario-1 pcap with no rule changes:

Run	9000002 alerts	9000002 timestamps
baseline (b7d9661)	3	18:04:54.946 · 18:05:22.045 · 18:05:33.102
verify (run 1)	2	18:05:45.276 · 18:06:05.317
verify-2 (run 2)	3	18:04:54.872 · 18:05:33.100 · 18:06:10.387
verify-3 (run 3)	3	18:04:54.522 · 18:05:24.903 · 18:05:24.875
verify-4 (run 4)	2	18:04:54.523 · 18:05:17.372

Counts oscillate 3, 2, 3, 3, 2 (3/5 fire three alerts; 2/5 fire two).

Timestamps shift across runs even when the count matches the baseline — same input, same rules, but different trigger points each time.

Smoking-gun evidence within a single run: run 3 fires 9000002 at 18:05:24.903909 and 18:05:24.875207 — **28 milliseconds apart**, both from the same source IP, both within the 10-second track by_src suppression window the rule's type both mode is supposed to enforce. Two alerts firing 28 ms apart from the same source proves Suricata's threshold-suppression state machine has a race condition under multi-threaded evaluation. The engine reports Threads created -> RX: 1 W: 12 FM: 1 FR: 1 — 12 worker threads racing on the shared threshold state.

ICS-detection-design implication: rate-threshold rules are not suitable as a sole detection signal where exact-count semantics matter (compliance reporting, forensic reconstruction, alert-dedup contracts). Cumulative flow tracking — counting unique destinations per source over a long window, without sub-second window arithmetic — is the more robust primitive for the same indicator class. It doesn't depend on per-thread state synchronization and doesn't suffer window-edge jitter.

Forward link: sub-session 3b/3c will redesign port-scan and connection-churn detection around flow-based primitives instead of rate thresholds. Cross-referenced in docs/phase2/detection/3a-recon-detection.md SID 9000002 discussion.

9. Suricata 8.0.5 modbus keyword has multiple silent comparator bugs; range form is the only reliable address filter (2026-05-24)

Source: Phase 2 sub-session 3b (Suricata Modbus protocol detection) — docs/phase2/detection/3b-modbus-protocol.md (shipped 5e06861). Six empirical keyword findings + two harness findings from four investigation cycles characterizing actual vs. documented behavior.

Summary: Suricata 8.0.5's modbus: rule keyword has working primitives (function N, access read|write <table>) and broken comparator forms. Characterized across all 6 rules in the 3b ruleset (SIDs 9000010-9000015) plus syntax variations on scenario-2's pcap.

Six keyword findings (headline only — full investigation in 3b evidence doc):

#	Keyword form	Behavior
K\$1	function N (exact FC match)	Works as documented
K\$2	access read\ write <table> (FC class match)	Works as documented
K\$3	address <N / address >N on access read rules	Silently no-op — clause parses but doesn't filter; rule fires regardless of address
K\$4	address <N on access write rules	Silently off-by-one — <3 matches addresses 0 and 1 only; address 2 silently excluded (would have missed scenario 3's alarm-coil attack)
K\$5	address X<>Y (range, exclusive)	Works AND has richer semantic than expected — matches if ANY address in the read range falls within [X,Y], not just the start address
K\$6	address N (exact match, N=0)	Produces zero matches; unverified for N≠0

Two harness findings:

#	Issue	Workaround
H\$1	In-container tcpdump -i any pcaps have empty TCP checksums (kernel offload); Suricata's default pcap-file mode rejects them from app-layer inspection	Per-run -k none flag for offline replay; keeps daemon defaults intact
H\$2	Mid-stream baseline pcaps (no TCP handshake captured) block app-layer parser engagement entirely — flow:established doesn't match, parser never sees reassembled bytes	Recapture from a fresh handshake (restart FUXA so its long-lived conn re-establishes mid-tcpdump); held item

Detection-design implications:

- Comparator forms (>N, <N) on read OR write rules cannot be trusted in 8.0.5. The 3b ruleset uses range form on reads (address 3<>65535 for out-of-map detection) and drops the address sub-clause on writes (source-IP + write-class is the robust primitive — no legitimate writer exists, so any non-FUXA write is suspicious regardless of address).
- **The general lesson:** when a keyword's documented behavior diverges from its actual behavior, use the robust subset of the keyword rather than work around bugs. Source-IP + protocol-direction filters are durable across Suricata versions; address comparators are advisory until verified per-version.
- **6/6 pcap predictions matched** in the final 3b validation run (251 alerts across 4 attack pcaps, 0 on baselines). The match arrived after four iteration cycles characterizing the keyword — the investigation IS the finding, not just the clean final result.

Forward link: if Suricata 8.0.6+ fixes the comparator bugs (K\$3, K\$4), the rules as shipped might suddenly over-fire on addr=3+ writes. The 3b evidence doc Decision §2 explicitly addresses this fragility — Path R (no address sub-clause) was chosen over Path Q (off-by-one compensation <4) on rule-honesty + upstream-fix-robustness grounds.

10. FUXA 1.3.1 ModbusTCP driver has no auto-reconnect; silent client drops require detection at the deployment layer (2026-05-24)

Source: Mid-Phase 2 sub-session 3b investigation. Driver source: `/usr/src/app/FUXA/server/runtime/devices/modbus/index.js` inside the `frangoteam/fuxa:1.3.1` container.

Summary: When FUXA's Modbus client TCP connection drops mid-poll, the driver logs the error and continues reading on the dead socket without retrying. No on-error reconnect handler exists. Comparison with sibling drivers in the same FUXA version: MQTT driver has `client.on("reconnect", ...)` handler; Redis driver has `reconnectStrategy: (retries) => Math.min(100 + retries * 200, 3000)` backoff; Modbus driver has neither. The device-config schema does not expose a reconnect-related field because the driver doesn't implement one.

Observed failure mode (2026-05-24): FUXA's HMI tank stopped rendering in the browser. `docker compose ps` showed `otlab-fuxa` as Up. FUXA logs showed zero Modbus error messages. `/proc/net/tcp` grep on the openplc side showed zero ESTABLISHED conns to :502 from FUXA's IP. Recovery: `docker compose up -d --force-recreate fuxa` re-established the connection and the HMI resumed rendering.

Mitigation (shipped 626fc56): Docker healthcheck on the fuxa service that probes `/proc/net/tcp` inside the FUXA container for an ESTABLISHED conn (state 01) to openplc:502 on either bridge IP (172.18.0.2 or 172.19.0.2 in little-endian hex). After 3 consecutive 30s-interval failures, Docker marks fuxa (unhealthy). Detection-only — no autoheal sidecar. **On unhealthy: operator runs `docker compose up -d --force-recreate fuxa`.** End-to-end verified (stop openplc → fuxa flips unhealthy at t=93s → restart openplc + recreate fuxa → healthy in 10s). Phase 1 attacker + Suricata services not disturbed by the test cycle.

Known Risk for production-equivalent deployments: detection-only healthcheck is sufficient for this lab's local use; production-grade autoheal (sidecar container monitoring health-status events + auto-restarting unhealthy containers) is a deferred follow-up for the

portfolio-prep phase post-cert. If FUXA Modbus drops become frequent (>1/week observed), investigate environmental trigger before adding autoheal.

11. 3c offline replay confirms pre-3c ruleset already detected scenario 5; new SIDs add forensic granularity, not detection coverage (2026-05-25)

Source: Phase 2 sub-session 3c (Suricata replay-attack detection) — docs/phase2/detection/3c-replay-attack.md (shipped 04dd25d, this session). Offline replay against captures/phase2-scenario-5-replay-2026-05-22.pcap; eve.json + fast.log retained at suricata/logs/replay-3c/ (gitignored).

Summary: Two new SIDs ship in 3c — 9000020 (FC 1 read coils from non-FUXA source) and 9000021 (FC 3 read holding from non-FUXA source). Both are protocol-aware companions to 3b's 9000012 / 9000013 (non-FUXA writes), completing the source-IP-filtered by-table matrix.

Validation produced 6 alerts against scenario-5 pcap, matching prediction exactly (all 4 per-SID counts hit):

- 9000003 x3 (attacker SYN to :502 from non-FUXA, three flows)
- 9000013 x1 (mutated FC 6 write, TID 0x00a3, value 4242)
- 9000020 x0 (no FC 1 from attacker in this pcap — verification gap held; positive exercise of 9000020 is a future-scenario item)
- 9000021 x2 (verbatim FC 3 replay TID 0x00a3 + verify-read TID 0x0042)

The pre-3c ruleset (11 SIDs) already produced 4 of those 6 alerts. The 9000020 / 9000021 additions contribute 2 alerts of forensic context (app_proto: modbus + decoded operation in the alert payload) but DO NOT change the answer to “did Suricata detect this attack.” The answer was yes pre-3c.

The 3c contribution is alert *richness*, not detection *coverage*.

Implication for downstream SIEM/SOC consumers: 9000021 fires twice per attack session (no per-session dedupe — one alert per non-FUXA read PDU). A multi-PDU read exfiltration would produce N alerts, not 1. Expected Suricata signature behavior; documented because alert volume per attack session matters for triage cost projections.

Forward link from §7: the protocol-level finding (no authentication, no integrity) shipped at scenario-5 documentation time. §11 is the detection-side closure: source-IP + read-class is the only Modbus-layer signal that operationalizes §7's “source IP is the only thing that distinguishes a clever attacker from FUXA” conclusion. Segmentation remains the actual control (§7 + §11).

Forward link from §8 — NOT REALIZED in 3c: §8 anticipated that “sub-session 3b/3c will redesign port-scan and connection-churn detection around flow-based primitives instead of rate thresholds.” 3c did not undertake this redesign. Rate-threshold rules 9000002 and 9000005 ship in 3c unchanged. Held as a future task — likely needs host-mode live capture to exercise threshold-state under load, so coupled to the Docker Engine on Linux migration (Finding §12). Not a 3c blocker.

12. Docker Desktop's host networking is layer-4 only on every platform; AF_PACKET live capture requires Docker Engine on Linux (2026-05-25)

Source: Phase 2 sub-session 3c live-capture evaluation — docs/phase2/detection/3c-replay-attack.md Live-Capture Finding §1 and Architectural Decision §1. Triggered by the question “can we enable live Suricata AF_PACKET capture in 3c to validate alerting on fresh attacks?” Answer required reading the Docker Desktop docs: <https://docs.docker.com/engine/network/drivers/host/>

Summary: Docker Desktop’s `network_mode: host` operates at layer 4 on every platform it ships — Windows, macOS, and Linux. This is a Docker Desktop architectural property: a Docker Desktop container runs inside a Linux VM (the Moby VM) separated from the host kernel by a virtualization boundary, regardless of the host OS. From the upstream docs:

“The host network feature of Docker Desktop works on layer 4. This means that unlike with Docker on Linux, network protocols that operate below TCP or UDP are not supported. Processes inside the container cannot bind to the IP addresses of the host because the container has no direct access to the interfaces of the host.”

“Docker on Linux” in that quote means Docker Engine (`docker-ce`) installed natively on a Linux host — NOT Docker Desktop for Linux, which still uses the Moby VM and therefore still has the L4 limitation. Docker Engine on Linux has no intermediating VM; host mode there is full L2 access.

Suricata’s af-packet capture requires raw L2 interface access. Docker Desktop’s host mode does not provide this. The `br-*` bridge interfaces visible from inside the Moby VM are not accessible to a host-mode Suricata container for AF_PACKET capture — different mechanisms, not the same as host mode on Docker Engine for Linux.

Implication: the obvious solution for the Docker-bridge-MAC-learning visibility limitation (3a/3b deferred live capture for this reason) does not exist on Docker Desktop, regardless of host OS. Three workarounds were evaluated for a future migration to Docker Engine on Linux:

1. `network_mode: host` on Docker Engine for Linux (`docker-ce` installed natively — inside WSL2 alongside or in place of Docker Desktop, OR on a dedicated Linux host) — works, simplest. Recommended path.
2. `macvlan` driver swap — works on either Docker Desktop or Docker Engine, but invasive compose rewrite and ties the lab to a specific host NIC name. Not recommended.
3. `iptables NFLOG` mirroring on the host kernel — only meaningful on Docker Engine for Linux (the Moby VM’s kernel isn’t directly accessible from Docker Desktop on the host side). Adds host-kernel state coupling and brittleness.

None of the three are 3c-actionable on the current Docker Desktop deployment. 3c ships with offline pcap replay only (Finding §11). Live-capture deployment is a post-cert portfolio-prep item, NOT a Phase 2 blocker.

Forward link: when this lab migrates from Docker Desktop to Docker Engine for Linux as a portfolio-prep step, host-mode Suricata becomes the canonical deployment shape. At that point, 3c’s offline-replay results should be re-validated against live traffic to confirm parity — same SIDs, same counts (or document divergence). Rate-threshold rules per §11’s forward-link-not-realized note will also need live-capture to exercise.

13. OpenPLC v3 archived upstream 2026-04-04; Phase 3 stack locked via ADR 0001 (Gate 2 closure, 2026-05-25)

Source: Tool-selection-rule retroactive evaluation — docs/decisions/0001-phase3-stack.md (this session). Triggered by the mid-Phase-2 codification of the tool-selection gate (≥ 3 candidates per role, deprecation status checked, platform-fit verified) applied retroactively to the foundational FUXA + OpenPLC v3 stack.

Summary: OpenPLC v3 was formally **archived** on github.com/thiagoralves/OpenPLC_v3 on 2026-04-04 with an explicit “End of Life — replaced by OpenPLC Runtime v4” notice. This is a stronger status than Phase 1’s “upstream EOL” framing: the repo is read-only, no future patches even hypothetically. The tool-selection rule’s “upstream-deprecated = hard stop unless explicit justification” clause therefore disqualifies v3 for Phase 3 and beyond.

ADR 0001 evaluated 5 PLC candidates and 6 HMI candidates with platform-fit verification on Docker Desktop on Windows + WSL2, plus two new ADR columns (Phase 2 finding regression risk; IDS rules regression risk). Outcome:

- **PLC: OpenPLC v4 (Autonomy-Logic).** Replaces v3. Image at ghcr.io/autonomy-logic/openplc-runtime:latest (NOT Docker Hub), pulled and characterized empirically this session. Beremiz dismissed on platform-fit grounds (wxPython IDE + Pyro RPC; no documented Docker split-deployment recipe for Windows + WSL2).
- **HMI: FUXA 1.3.2.** Bumped from 1.3.1 (compose tag-only change). Same vendor, minor version, released 2026-05-19. Alternatives dismissed on regression-risk + migration-cost grounds — switching HMIs would force re-baselining all 13 SIDs and re-shaping 5 attack scenarios, and no candidate (Ignition Maker, Scada-LTS, Node-RED Dashboard 2.0) is sufficiently better for the security-lab role to justify that.

Implication: Phase 3 starting task is migration. Not done this session — this ADR commits to the destination; the migration itself is Phase 3 work. Findings §3 (web UI), §4 (silent-zero out-of-map), §5–§7 (write semantics + reassertion), §10 (FUXA reconnect) all need re-validation against the new runtime, per ADR Consequences.

Sources: ADR docs/decisions/0001-phase3-stack.md; archive notice github.com/thiagoralves/OpenPLC_v3; v4 home github.com/Autonomy-Logic/openplc-runtime; FUXA 1.3.2 release github.com/frangoteam/FUXA/releases/tag/v1.3.2.

14. OpenPLC v4 ships Modbus as a plugin that is silent until drivers.cfg is supplied AND PLC enters RUN state (empirical, 2026-05-25)

Source: ADR 0001 empirical verification step — pulled ghcr.io/autonomy-logic/openplc-runtime:latest (356 MB, amd64, `entrypoint bash ./start_openplc.sh`) and characterized out-of-box behavior on Docker Desktop on Windows + WSL2.

Summary: Three layered conditions gate Modbus listening on v4, not the single condition v3 had (“container up = :502 listening”):

1. **Image EXPOSE list** contains only 8443/tcp. Port 502 is not in the EXPOSE manifest at all.
2. **Plugin enablement.** The image ships `/workdir/venvs/{modbus_slave,modbus_master,opcua,runtime}` Python `venvs` and plugin source under

/workdir/core/src/drivers/plugins/python/modbus_slave/, but **no drivers.cfg file exists** anywhere in the freshly-pulled image. Modbus is therefore installed but unconfigured.

3. **PLC state.** Even with a hypothetical drivers.cfg in place, the Phase 2 spike (2026-05-21) observed that Modbus does not begin listening until the PLC enters RUN state, which requires a compiled program uploaded via the v4 REST API. That spike observation is consistent with — though not independently re-verified against — current v4.0.9.

Operational consequence for Phase 3 lab bring-up: the sequence becomes (a) docker compose up → REST API on 8443 only, (b) operator-supplied drivers.cfg + program upload via Editor → (c) PLC RUN state → (d) Modbus on :502 finally listens. This is more operator steps than v3 required and must be documented in the Phase 3 operator runbook (separate Phase 3 artifact).

Implication for ADR 0001's "Migration cost" column: the column score for OpenPLC v4 was "Substantial" precisely because of this finding — the operator workflow changes, not just the compose tag. Anyone proposing a simpler "swap the image tag and you're done" migration plan in the future should be pointed at this finding to recalibrate.

Sources: ADR docs/decisions/0001-phase3-stack.md Empirical verifications section; v4 docs github.com/Autonomy-Logic/openplc-runtime/blob/main/docs/DOCKER.md.

Phase 3 Findings

Phase 3 — PLC migration from OpenPLC v3 to v4 — surfaced findings §13-§23 across 2026-05-25 (Gate 2 + Stops 1-2) and 2026-05-26 (Stops 3-6). §13 and §14 landed in the Phase 2 Findings section above at commit bf69781 (Gate 2 ADR shipping); §15-§23 below are dated 2026-05-26. All findings are append-only; corrections to earlier findings appear as inline acknowledgements in later findings, never as retroactive rewrites.

15. OpenPLC v4 Modbus listening is a four-gate dependency chain (2026-05-26)

Source: Phase 3 Stop 1 + Stop 2 empirical work — ghcr.io/autonomy-logic/openplc-runtime images v4.1.0-rc.1 and v4.1.0 pulled and characterized; logs captured during plugins.conf authoring + first program upload.

Summary: The Phase 2 v4 spike (2026-05-21) framed v4's Modbus listener as gated on two conditions ("plugins.conf supplied AND PLC in RUN state"). Empirical characterization showed **four gates**:

1. plugins.conf exists at /workdir/plugins.conf with a modbus_slave row enabled
2. The plugin's config.json (referenced from plugins.conf column 5) overrides modbus_slave plugin's default port from 5020 → 502
3. A compiled libplc_*.so exists in /workdir/build/ (output of the runtime's compile step, triggered by POST /api/upload-file accepting a Editor-produced bundle)
4. PLC is in RUN state (transitioned via authenticated GET /api/start-plc, which itself requires gate 3 satisfied)

With gates 1+2 only, runtime logs [ERROR] No libplc_*.so file found in ./build → [ERROR] Failed to set PLC state to RUNNING; GET /api/status returns STATUS:EMPTY; GET /api/start-plc returns

START:ERROR; port 502 stays silent. Verified empirically against v4.0.9 and v4.1.0.

Correction to §13/§14 framing: the runtime config file is `plugins.conf`, NOT `drivers.cfg` as §13/§14 described. Load path is `./plugins.conf` relative to `/workdir`, per `plc_main.c:108` (v4.0.9) / `:110` (v4.1.0) and `plc_state_manager.c:246` / `:254`. The earlier “drivers.cfg” terminology was carried in from the original v4 spike notes; actual source uses `plugins.conf`.

Cross-refs: `docs/decisions/0001-phase3-stack.md` Corrections forward; `plc/v4/plugins.conf` and `plc/v4/modbus_slave_config.json` (lab artifacts).

16. OpenPLC Editor v4 desktop application is an architectural prerequisite, not optional (2026-05-26)

Source: Phase 3 Stop 2/3 — empirical attempt to upload `tank_fill.st` via `POST /api/upload-file` failed with the runtime requiring a ZIP of post-MATIEC C files, not raw `.st`. Runtime source confirmed.

Summary: The v4 runtime is genuinely headless — no MATIEC compiler ships in the runtime image. Editor README line 66 (verbatim): “*Compile in Editor - The Editor compiles locally (JSON → XML → ST → C files) and packages sources into program.zip*”. `POST /api/upload-file` expects a ZIP of post-MATIEC C files (`Config0.c`, `Res0.c`, `debug.c`, `glueVars.c`, `lib/`), not raw `.st`. Editor lives at `github.com/Autonomy-Logic/openplc-editor` (v4.1.4 installed during this session).

Implication for ADR 0001 “Migration cost” column: the ADR said “Editor is a separate desktop application (wxPython).” This understated the requirement — Editor is the *mandatory* client-side half of the compile toolchain. The runtime cannot accept raw ST source. **Operators must install the Editor on Windows as a non-negotiable lab bring-up step.** This is captured in the ADR’s Corrections forward.

Note on the Editor’s stack: v4.1.4 is Electron, not wxPython (the ADR’s “wxPython” framing was carried forward from Beremiz comparison and was incorrect for v4 Editor). Installer is at `github.com/Autonomy-Logic/openplc-editor/releases`.

17. OpenPLC v4 modbus_slave plugin default port is 5020, not 502 (2026-05-26)

Source: Phase 3 Stop 2 — read of `simple_modbus.py:983` `gPort = 5020` inside the runtime image. Verified on both v4.0.9 and v4.1.0 images.

Summary: The `modbus_slave` Python plugin’s default Modbus port is **5020**, not the standard 502. Standard-port operation requires explicit override via the plugin’s `config.json` setting `network_configuration.port = 502`. This default is not documented in the upstream README; only visible by reading the plugin source.

Lab `plc/v4/modbus_slave_config.json` ships this override:

```
{ "network_configuration": { "host": "0.0.0.0", "port": 502 } }
```

The config file path is set in `plugins.conf` column 5 (`plugin_related_config_path`), resolved relative to the runtime’s CWD (`/workdir`).

18. Editor v4.1.x and Runtime v4.1.x are not wire-compatible (2026-05-26)

Source: Phase 3 Stop 3 — empirical attempt to upload an Editor v4.1.4 MatIEC bundle to runtime v4.1.0 rejected with explicit pipeline-mismatch error. Source code confirmed.

Summary: The v4.1.x **Runtime** ships a code-gen pipeline change (“STruC++”) that rejects MatIEC artifacts. The v4.1.x **Editor** still produces MatIEC artifacts. The two are not wire-compatible despite carrying the same major-minor version.

Empirical evidence:

- Editor v4.1.4 src/main/modules/compiler/compiler-module.ts invokes iec2c (MatIEC) at line 433. The OpenPLC Runtime v4 target branch at lines 1727-1755 adds conf/*.json files to the bundle but does NOT change the compiler. Full-tree code search for strucpp returns **zero matches**.
- Runtime v4.1.0 scripts/compile.sh rejects bundles containing Config0.c/glueVars.c: *“core/generated contains MatIEC files. This runtime no longer supports MatIEC programs. Re-export the project from a STruC++-aware editor build.”* The webserver/restapi.py:38 docstring describes the design intent. Full-tree code search for strucpp in the runtime repo also returns **zero matches**.

The runtime’s error message references a “STruC++-aware editor build” that does not exist in any tagged Editor release.

Operator workaround applied: pin runtime to ghcr.io/autonomy-logic/openplc-runtime:v4.0.9 — last MatIEC-accepting final release (2026-03-05). v4.0.9 also emits literal X-OpenPLC-Runtime-Version: v4 (vs v4.1.x’s variable header), which conveniently sidesteps the separate Editor literal-compare bug (§19).

Correction to ADR 0001 “version pin”: ADR chose “OpenPLC v4 (Autonomy-Logic)” without specifying patch. Empirical landing is **v4.0.9 pinned**; v4.1.x is currently the broken combination. Captured in ADR Corrections forward.

Upstream issue filed: github.com/Autonomy-Logic/openplc-editor/#782.

19. Editor v4.1.4 validateRuntimeVersion literal-compare bug (2026-05-26)

Source: Phase 3 Stop 3 — Editor’s “Runtime Version Mismatch” popup observed against v4.1.0 runtime; source code traced.

Summary: src/utils/device.ts in Editor v4.1.4 has a literal-string-compare bug in its runtime version-gate. getExpectedRuntimeVersion() regex-extracts major-only (“v4”) from the device target string. validateRuntimeVersion() then does literal normalizedDetected !== expectedVersion against the **un-normalized** full runtime header. Any runtime emitting a header more specific than “v4” (e.g., “v4.1.0”) trips a spurious mismatch popup at board.tsx:236, sets connectionStatus='error', and disables the Play button.

Operator workaround: runtime v4.0.9 emits literal “v4” (hardcoded in restapi.py:38), which matches the expected. v4.1.x runtimes emit the full release tag via the RUNTIME_VERSION build-time variable, which trips the bug. Combined with §18, v4.0.9 is the runtime pin that works around both upstream bugs.

Upstream issue filed: [github.com/Autonomy-Logic/openplc-editor#781](https://github.com/Autonomy-Logic/openplc-editor/issues/781).

20. Runtime's update_plugin_configurations disables plugins on bundle upload — read-only mount is load-bearing (2026-05-26)

Source: Phase 3 Stop 3 first successful upload — runtime logs captured at 2026-05-26 16:43 UTC.

Summary: Runtime's `handle_upload_file` calls `update_plugin_configurations(extract_dir)` which reads the existing `plugins.conf`, inspects the uploaded bundle for a `conf/` directory, and **disables every plugin** that doesn't have a matching `conf` file in the bundle. It then attempts to write the modified `plugins.conf` back to disk.

The lab's `:ro` bind mount on `/workdir/plugins.conf` makes the write fail:

```
[2026-05-26 16:43:24] [ERROR] Failed to save plugin configuration to
plugins.conf: [Errno 30] Read-only file system: 'plugins.conf'
172.18.0.1 - - [26/May/2026 16:43:24] "POST /api/upload-file
HTTP/1.1" 200 -
[2026-05-26 16:43:25] [INFO] Plugin modbus_slave started
successfully
[2026-05-26 16:43:25] [INFO] PLC State: RUNNING
```

This failure is causally load-bearing for the lab. The disable-decision-then-write logic doesn't commit the in-memory disabled state until the write succeeds. Because the write fails before commit, `modbus_slave` stays enabled in the runtime's view, and is started successfully when the PLC enters RUN. **If the bind mount were `:rw`, `modbus_slave` would persist as disabled and port 502 would be silent.**

Forward fix to remove dependency on this accident: populate the Editor project's servers section (Project Tree → Servers → add Modbus TCP server, port 502) so the bundle includes `conf/modbus_slave.json`. Then `update_plugin_configurations` keeps `modbus_slave` enabled by intent rather than by write-failure. Until that's authored in the Editor project, `:ro` discipline on `plugins.conf` is mandatory for the lab to function — not optional.

21. FUXA 1.3.2 plugin-architecture change breaks pre-1.3.2 project DBs (2026-05-26)

Source: Phase 3 Stop 4 attempt — FUXA 1.3.2 image pulled and run against the same `project.fuxap.db` that loaded cleanly on 1.3.1. Empirical log evidence captured.

Summary: FUXA 1.3.2 introduces an undocumented plugin-architecture change that breaks devices defined in pre-1.3.2 project DBs. Same `project.fuxap.db` (device OpenPLC, type ModbusTCP) loads cleanly on FUXA 1.3.1 but fails on 1.3.2 with:

```
2026-05-26T16:55:52.637Z [WAR] try to create OpenPLC but plugin is
missing!
```

(Version line in same logs: FUXA V.1.3.2-2827; matching 1.3.1 attempt: FUXA V.1.3.1-2789 followed by 'OpenPLC' created + 'OpenPLC' start cleanly.) Device was never instantiated on 1.3.2 → no Modbus polling attempted → no ESTABLISHED conn in `/proc/net/tcp` → healthcheck failed.

The “plugin is missing” warning text references the **device name** (OpenPLC) rather than the missing **plugin type** (ModbusTCP driver), which obscures the root cause; the actual missing component is the ModbusTCP driver presumably moved to a separate npm package.

Operator workaround applied: pin FUXA at 1.3.1. Gate 2 ADR’s HMI decision rationale was “same vendor minor version bump” — the *continuity* signal, not a specific 1.3.2 dependency. Staying on 1.3.1 satisfies the ADR’s intent. Captured in ADR Corrections forward.

Upstream issue NOT YET filed (Phase 3 Stop 6 outcome — staged in `~/ .claude/projects/<project>/memory/phase3-migration-progress.md` Path Z section for filing at `github.com/frangoteam/FUXA/issues` later). file:line evidence in FUXA 1.3.2 source not yet collected; deferred to issue filing time.

22. FUXA → openplc routing now rides ot_zone exclusively (2026-05-26)

Source: Phase 3 Stop 4 cutover — `/proc/net/tcp` inspection inside FUXA after pointing the device DB at `openplc_v4:502`.

Summary: FUXA→openplc Modbus polling rode `mgmt_zone` in the v3 era (Phase 2 Finding §2 — Docker DNS happened to resolve `openplc` to the `mgmt_zone` IP first). Post-Stop 4, FUXA polls `openplc_v4:502`. Since `openplc_v4` is attached to `ot_zone` only (`mgmt_zone` attachment intentionally deferred — see ADR Corrections forward), Docker DNS resolves to 172.18.0.5. FUXA’s ESTABLISHED Modbus connection is now:

```
src 030012AC:*      (172.18.0.3:* FUXA on ot_zone)
dst 050012AC:01F6   (172.18.0.5:502 openplc_v4 on ot_zone)
state 01           (ESTABLISHED)
```

Implication for the lab’s network architecture: `ot_zone` is now genuinely the “process network” carrying Modbus traffic — closer to the architectural intent than the v3-era accidental `mgmt_zone` routing. The healthcheck IP hex was updated accordingly in `docker-compose.yml` (050012AC:01F6).

Implication for Suricata: `$OPENPLC_IP` was updated to [172.18.0.5] in Stop 6 cleanup (removing the v3 IPs that the rollback-safety-net period required). `$FUXA_IP` retains [172.18.0.3, 172.19.0.3] — FUXA’s IP didn’t change, only the routing direction.

23. Phase 3 Stop 5 Suricata SID re-baseline: zero regressions, single-variable fix (2026-05-26)

Source: Phase 3 Stop 5 — all 13 Suricata SIDs (9000001-21 in the documented ranges) replayed against fresh v4-targeting attack pcaps captured from the temporarily-`ot_zone`-attached attacker container.

Summary: All 13 SIDs validated against the v4 + FUXA-1.3.1 stack with **zero coverage regressions, zero false positives, zero rule retirements**. The migration’s IDS impact was a single variable expansion: `$OPENPLC_IP` expanded from [172.18.0.2, 172.19.0.2] to [172.18.0.2, 172.19.0.2, 172.18.0.5] during the coexistence window, then reduced to [172.18.0.5] in Stop 6 when v3 was removed from compose. **No rule text changes, no SID retirements.**

Per-scenario results (live attacks captured from attacker, offline-replayed through Suricata 8.0.5):

- **Scenario 1 (recon):** 9000001 ×1, 9000002 ×1, 9000003 ×1, 9000004 ×5. 9000005 (RST burst ≥15/10s) did not fire — same as

Phase 2 against v3 (known non-determinism per Finding §8). 4 of 5 recon rules confirmed against v4-targeting traffic.

- **Scenario 2 (Modbus FC scan):** 9000003 ×1, 9000010 ×1, 9000011 ×1, 9000012 ×2, 9000013 ×2, 9000014 ×2, 9000015 ×2, 9000020 ×2, 9000021 ×3. All 9 Modbus rules fire correctly on v4 IPs.
- **Scenario 3 (alarm coil burst, 60 writes):** 9000003 ×1, **9000012 ×60. Exact match to Phase 2 count.**
- **Scenario 4 (HR writes, 60 + 60 + 1):** 9000003 ×1, **9000013 ×121. Exact match to Phase 2 count.**
- **Scenario 5 (replay attack):** 9000003 ×3, 9000013 ×1, 9000021 ×2 = 6 alerts. **Exact match to Phase 2 step 3c offline-replay against the original scenario-5 pcap.**

Plus a protocol-level confirmation from scenario 5: v4 mutated-write of HR 1 = 4242 was overwritten back to 20 within ~1 scan, identically to v3. **v4's §5.3 reassertion property holds.** Same wire protocol, same detection signal, same defensive properties.

Live captures retained at captures/phase3-stop5-s{1..5}-*.pcap (gitignored). Suricata replay output at suricata/logs/stop5-s{1..5}/ (gitignored).

Stop 6 cleanup applied: \$OPENPLC_IP reduced to [172.18.0.5]; \$OPENPLC_WEB_PORTS reduced to [8443] (v4 has no web UI on 8080). The Gate 2 ADR's "IDS rules regression risk: Moderate" column for OpenPLC v4 — empirical result is **Minimal** (variable scope only).

Queue Item 1 Findings

Queue Item 1 — the historian + visualization + collector stack (InfluxDB v2 + Telegraf + Grafana) — completed 2026-05-27 in commit 764dacc. ADR is docs/decisions/0002-historian-stack.md. Findings §24–§29 below were surfaced during the build (Stops 1–5) and document the gotchas captured for future runbook updates and Phase 4+ tool-selection work.

24. InfluxDB v2 Docker latest tag flipped to v3 Core on 2026-05-27; pinning is now load-bearing in a way it normally isn't (2026-05-27)

Source: ADR 0002 verification step (2026-05-27) — Docker Hub https://hub.docker.com/_/influxdb and the InfluxData install doc <https://docs.influxdata.com/influxdb/v2/install/> both surface the flip explicitly. The install doc shows the literal banner: *"On May 27, 2026, the latest tag for InfluxDB Docker images will point to InfluxDB 3 Core."*

Summary: On 2026-05-27 the official influxdb image's latest tag stopped pointing at v2 and started pointing at v3 Core. v3 is not a minor version bump — it is a different product:

- Apache Arrow + Parquet storage engine (vs v2's Time-Structured Merge Tree)
- No Flux query language (SQL-first via Flight SQL only)
- 72-hour default single-query range cap in Core (config-raisable; lifted entirely only in Enterprise)
- Larger RAM footprint at idle and during compaction (per QuestDB-published benchmark of the v3 Core alpha)

ADR 0002 pinned influxdb:2.9.1 explicitly to protect against this. This finding documents the flip as the reason — so anyone reading the compose file later understands why the pin is not stylistic.

Implication: Any compose snippet, blog post, or AI completion that says `image: influxdb:latest` from 2026-05-27 forward will silently land on v3 Core. For this lab the difference is a different storage engine, no Flux, and the 72-hour query cap — none of which matches what the Telegraf integration or future Grafana dashboards assume. Pinning `:2.9.1` is the only safe shape.

Sources: ADR 0002 docs/decisions/0002-historian-stack.md;
<https://docs.influxdata.com/influxdb/v2/install/>;
https://hub.docker.com/_/influxdb.

25. Telegraf is the only OSS collector with first-class Modbus input + native InfluxDB output (2026-05-27)

Source: ADR 0002 collector-role evaluation — docs/decisions/0002-historian-stack.md “Collector role” section. Reference finding for future tool-selection work.

Summary: Telegraf 1.38’s `inputs.modbus` plugin is the only OSS collector with a first-class, actively-maintained Modbus input plugin paired with a native InfluxDB output. The closest alternatives — Node-RED + `node-red-contrib-modbus`, custom Python with `pymodbus`, Apache NiFi `GetModbusTCP` — are either heavier-weight (NiFi JVM footprint; NiFi processor is contributed-non-core) or impose more code/flow authoring than the role warrants (Node-RED flow plumbing; `pymodbus` is a library, not a daemon). None of them is a serious peer for the specific PLC → InfluxDB job. Telegraf’s `inputs.modbus` + `outputs.influxdb_v2` is a zero-configuration handshake (set URL, token, org, bucket — done).

Implication: For Phase 4+ tool selection decisions involving Modbus collection, Telegraf is the default. Switching away should require an explicit justification, the same way the tool-selection discipline established in Phase 2 requires for any deprecated-upstream substitution. The structural reason — first-party paired plugins, not glue code — is durable across Telegraf versions.

Sources: ADR 0002 docs/decisions/0002-historian-stack.md Collector role; Telegraf modbus plugin README
<https://github.com/influxdata/telegraf/blob/v1.38.4/plugins/inputs/modbus/README.md>.

26. Telegraf 1.38 modbus plugin `byte_order` requires 4-character form even for 16-bit fields (2026-05-27)

Source: Queue Item 1 Stop 4 — Telegraf container crashlooped immediately after first start:

```
E! [telegraf] Error running agent: could not initialize input
  inputs.modbus: configuration invalid for device "openplc_v4":
    unknown byte-order "AB"
```

Summary: The Telegraf 1.38 modbus plugin’s `request-style` configuration accepts only four `byte_order` values: `ABCD`, `DCBA`, `BADC`, `CDAB`. Two-character forms (`AB`, `BA`) are rejected. The plugin uses the same 4-character form for all field widths — only the first two characters are semantically meaningful for 16-bit fields, but the literal 4-character value is still required syntactically.

For OpenPLC’s `modbus_slave` plugin (which writes Modbus-standard big-endian), the correct value is `byte_order = "ABCD"` regardless of whether the field is `INT16` or `INT32`.

Initial Stop 2 draft of telegraf/telegraf.conf used byte_order = "AB" based on incorrect intuition about half-of-ABCD for 16-bit fields. Caught at runtime via container crashloop; verified against the v1.38.4 README which states the 4-character forms are the only accepted values.

Implication: For any future Telegraf modbus config in this lab, byte_order = "ABCD" is the canonical big-endian value. Coil and discrete-input fields do NOT need a byte_order at all — the README's *2 footnote documents that the field is ignored for those register types. Lesson generalised: when verifying ONE field in a plugin config against upstream source, verify EVERY field in the same block in the same lookup pass — cheaper than catching a second mismatch at runtime.

Sources: Telegraf modbus plugin README

<https://github.com/influxdata/telegraf/blob/v1.38.4/plugins/inputs/modbus/README.md>; lab artifact telegraf/telegraf.conf (commit 764dacc).

27. Docker Desktop on Windows + WSL2 caches bind-mount source paths in the container mount manifest — restart does NOT propagate host-file edits (2026-05-27)

Source: Queue Item 1 Stop 4 — applying the §26 byte_order fix to telegraf/telegraf.conf and trying to restart the container.

Summary: When a bind-mounted host file is edited while the container is running on Docker Desktop on Windows + WSL2, the change does NOT propagate to the container via docker compose restart <service>. Docker Desktop's WSL2 bind-mount mirror caches the resolved source path in the container's mount manifest (under /run/desktop/mnt/host/wsl/docker-desktop-bind-mounts/Ubuntu/<hash>). Restart reuses that stale manifest. Two failure modes:

1. **Silent staleness** — restart succeeds, the container starts, and reads the OLD file contents from the cached mirror. The host file change has no effect.
2. **Bind-mount race** — if Docker Desktop has garbage-collected the cached path, restart fails with runc create failed: error mounting ... no such file or directory on the bind-mount source. Sibling failure mode to the pre-existing "mkdir ... file exists" race during docker compose up that the operator runbook documents alongside this finding.

After applying the §26 fix to telegraf.conf on disk, the container continued to log the same unknown byte-order "AB" error on subsequent restart attempts. Plain docker compose restart did not propagate the change; docker compose up -d --force-recreate telegraf did. The bind-mount cache behavior described above is the most likely explanation; the recovery is documented regardless of root cause.

Recovery: docker compose up -d --force-recreate <service> re-resolves the bind-mount source against the current host file. Equivalent: docker compose down <service> followed by docker compose up -d <service>. Plain docker compose restart is insufficient for any bind-mount-backed config change on this platform.

Implication: Operator runbook needs an explicit "edit a bind-mounted config → must --force-recreate" entry. Now sibling content to the existing "mkdir ... file exists" bind-mount race in the runbook's troubleshooting section. Both are Docker Desktop / WSL2 mount-mirror behavior, not bugs in any specific container — they are structural to the platform.

Sources: Lab logs from Queue Item 1 Stop 4 (2026-05-27). Docker Desktop's WSL2 bind-mount documentation does NOT call out this caching behavior explicitly — the rule was derived empirically and is documented here as institutional knowledge.

28. Grafana v13 deprecated

/api/datasources/name/{name}/health — use

/api/datasources/uid/{uid}/health (2026-05-27)

Source: Queue Item 1 Stop 5 — datasource health verification step.

Summary: Grafana v13.0.1's HTTP API no longer routes `/api/datasources/name/{name}/health`. The name-based health endpoint returns 404 `{"message": "Not found"}` with the access log line `handler=notfound`. The path is not just unimplemented — it is actively unrouted. The UID-based endpoint at `/api/datasources/uid/{uid}/health` works as expected and returns the proper `{"status": "OK", "message": "datasource is working. N buckets found"}` payload for a working InfluxDB datasource.

The datasource UID is visible at `GET /api/datasources` (the list endpoint, which IS still routed). For provisioned datasources Grafana auto-generates the UID at startup; the value also appears in the provisioning log line: `msg="inserting datasource from configuration" name=<name> uid=<uid>`.

Implication: Any future operator playbook or smoke-test script that uses the name-based health endpoint will see 404 and may wrongly conclude provisioning failed. Runbook entries need to use the UID-based URL going forward, or fetch the UID first via the list endpoint.

Sources: Lab logs from Queue Item 1 Stop 5 (2026-05-27); Grafana 13 HTTP API docs
https://grafana.com/docs/grafana/v13.0/developers/http_api/data_source/.

29. Operational consequence of §15: docker compose down triggers a manual PLC re-upload via Editor v4 before Modbus listens again (2026-05-27)

Source: Queue Item 1 Stop 4 — Telegraf container went healthy but Modbus polls returned connection refused on 172.18.0.5:502 for ~3 minutes until the operator brought the PLC to RUN state through the Editor.

Summary: Finding §15 documents OpenPLC v4 Modbus listening as a 4-gate chain. Two gates (`plugins.conf` + `modbus_slave_config.json`) are bind-mounted from the host and survive any container lifecycle. Two gates (compiled `libplc_*.so` + PLC RUN state) live inside the container — the `.so` in `/workdir/build/`, the RUN state in-memory.

A `docker compose down` (or `down -v`) removes the container, which erases the in-container gates. On the next `docker compose up -d`, the runtime starts cleanly but logs `[ERROR] No libplc_*.so file found in ./build` and `[ERROR] Failed to set PLC state to RUNNING` — and port 502 stays dark. The bind-mounted config is fine; only the compiled program and RUN state are missing, and both require an Editor v4 upload + Compile + Run cycle that has not happened yet.

In contrast, `docker compose stop` pauses the container without destroying it. On `docker compose start` the compiled `.so` is still in `/workdir/build/` and the PLC transitions back to RUN automatically. No Editor action needed.

Practical rule: stop = no re-upload needed. down = re-upload needed. This is captured as a bold callout in the operator runbook §2 (Daily stop) as part of the Queue Item 1 runbook update.

Implication: For any operator-facing daily workflow this is a sharp distinction. The runbook calls out the rule explicitly so a first-time operator doesn't run down for routine maintenance and find themselves trying to diagnose a silent Modbus port when the real recovery is "go open the Editor and re-upload."

Sources: Finding §15 (the 4-gate chain this operationalizes); lab observation from Queue Item 1 Stop 4 (2026-05-27); operator runbook §2 + §3 (Queue Item 1 update, this commit).

Phase 2 Backlog (security operations on the Phase 1 stack — documented; not built)

- ☒ **Suricata passive monitoring** — sub-sessions 3a (recon detection, shipped 2026-05-23), 3b (Modbus protocol detection, shipped 2026-05-24), and 3c (replay-attack detection + offline validation closure, shipped 2026-05-25) all complete. Suricata 8.0.5 multi-homed on `ot_zone` + `mgmt_zone` with pinned IPs (172.18.0.4 / 172.19.0.4) and Modbus app-layer parser enabled. 13-rule ruleset total: 5 recon rules (SIDs 9000001-9000005, validated against scenario-1's pcap, 3/5 fired with full evidence) + 6 Modbus protocol rules (SIDs 9000010-9000015, 6/6 prediction match across 4 attack pcaps and 2 baselines) + 2 non-FUXA Modbus-read rules (SIDs 9000020-9000021, all 4 per-SID counts matched prediction against scenario-5 pcap; 9000020 not positively exercised — held as future-scenario verification item). Full write-ups in `docs/phase2/detection/3a-recon-detection.md`, `docs/phase2/detection/3b-modbus-protocol.md`, and `docs/phase2/detection/3c-replay-attack.md`. **Live-capture deployment deferred** to post-cert portfolio-prep work (per Finding §12: Docker Desktop host mode is L4-only on every platform; `AF_PACKET` requires Docker Engine on Linux).
 - ☒ **Attacker container in attacker_zone** (shipped 2026-05-22) — pinned Dockerfile on `debian:bookworm-20260518-slim` with `nmap`, `tcpdump`, `curl`, `dig`, `ping`, `ip`, `pymodbus 3.6.9`, `scapy`. Sits idle on `attacker_zone` only; reached via `docker exec`. Segmentation proof in [Phase 2 Findings §1](#).
 - ☐ Historian (InfluxDB + Grafana)
 - ☒ **Written-up attack scenarios** (shipped 2026-05-22) — five scenarios executed against the live stack from an `ot_zone`-attached attacker; six new findings logged (§2-§7 above). Per-scenario evidence in `docs/phase2/scenarios/*.md`; capture artifacts gitignored in `captures/`. Operational follow-up: add procs to `plc/Dockerfile` and `attacker/Dockerfile` so `pkill` is available without runtime install (currently installed at runtime per `docs/monitoring.md §3 Method B`; baked-in install removes the container-recreation loss step).
 - ☐ Real network diagram with assigned IPs
-

Phase 3 Backlog (platform migration and zone hardening — documented; not built)

- ☐ Tighten `mgmt_zone` ↔ `ot_zone` into a hard boundary (currently soft)
- ☒ **Migrate OpenPLC v3 → v4 — completed 2026-05-26 across Stops 1-6**. Runtime pinned to v4.0.9 (NOT v4.1.x — wire-

incompatible per §18). Editor v4.1.4 installed on Windows host (mandatory prerequisite per §16; not optional). FUXA pinned at 1.3.1 (1.3.2 broken per §21). All 13 Suricata SIDs validated against the new stack with zero regressions and a single-line \$OPENPLC_IP update (§23). Two upstream issues filed for the v4.1.x Editor→Runtime incompatibility and the Editor's literal-compare version-gate bug (Autonomy-Logic/openplc-editor#781, #782). v3 service removed from compose; v3 image kept on disk for forensic-only fallback. Full migration history + corrections-forward in docs/decisions/0001-phase3-stack.md.

Environment Prerequisite (MANUAL)

- Docker Desktop on Windows with **WSL2 integration enabled** for this distro. (*Status 2026-05-21: Docker Desktop installed, WSL2 integration confirmed. Lab proven running end-to-end — OpenPLC built from pinned source, ST compiled clean, PLC Running, live Modbus/TCP on :502 verified via pymodbus, FUXA connected with HMI animating, Wireshark baseline capture landed. Phase 1 Acceptance Gate fully met.*)